

Todo List

Aplicación Full Stack para examen técnico de Ola Tech Inc.

Documentación Técnica

1. Descripción General

Aplicación web de gestión de tareas con autenticación de usuarios. Permite registrarse, iniciar sesión con usuario/contraseña o con Google, y administrar tareas personales con fechas de vencimiento.

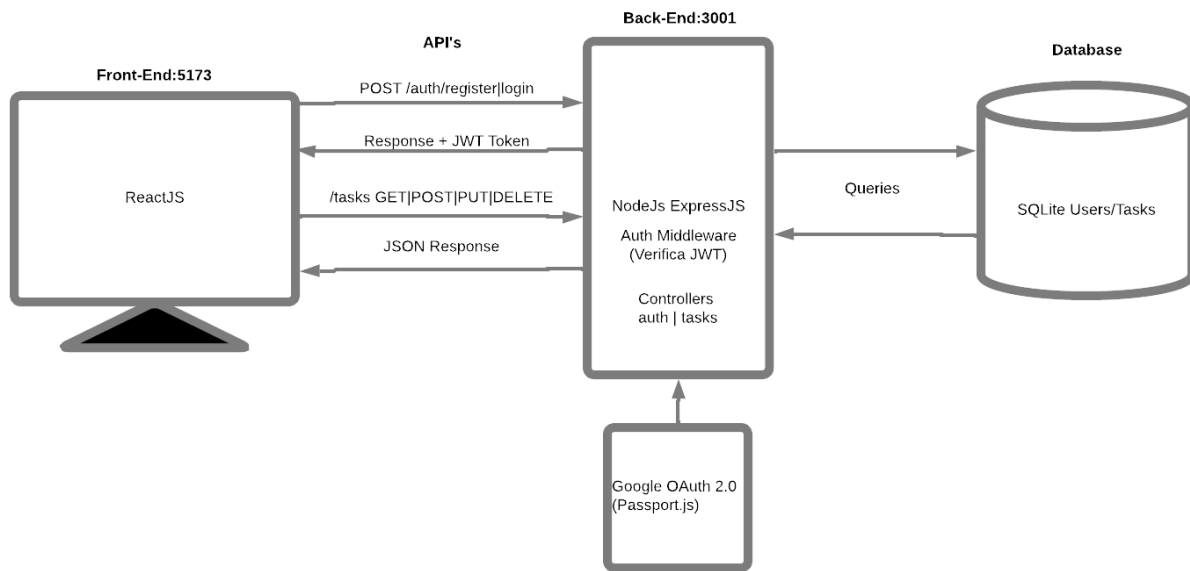
2. Stack Tecnológico

Capa	Tecnología
Frontend	React + Vite – diseño atómico
Backend	Node.js + Express
Base de datos	SQLite (better-sqlite3)
Autenticación	JWT + Google OAuth 2.0 (Passport.js)

3. Arquitectura General

El proyecto sigue una arquitectura cliente-servidor desacoplada. El frontend y el backend corren en puertos separados y se comunican exclusivamente mediante HTTP REST con JSON.

Diagrama de comunicación:



4. Modelo de Datos

Tabla users

Campo	Tipo	Descripción
id	INTEGER	Clave primaria, autoincremental
username	TEXT	Nombre de usuario único
password	TEXT	Contraseña hasheada (bcrypt) – null para Google
google_id	TEXT	ID de Google OAuth – null para usuarios locales

Tabla tasks

Campo	Tipo	Descripción
id	INTEGER	Clave primaria, autoincremental
user_id	INTEGER	FK → users.id (ON DELETE CASCADE)
text	TEXT	Descripción de la tarea
completed	INTEGER	0 = pendiente, 1 = completada
due_date	TEXT	Fecha límite (opcional)

Relación: cada tarea pertenece a un usuario. Si el usuario es eliminado, sus tareas se eliminan en cascada (ON DELETE CASCADE).

5. API REST

Autenticación – /auth

Método	Ruta	Descripción	Auth
POST	/auth/register	Registro con usuario y contraseña	No
POST	/auth/login	Login local, retorna JWT	No
GET	/auth/google	Inicia flujo OAuth con Google	No
GET	/auth/google/callback	Callback de Google, redirige con token	No

Tareas – /tasks

Método	Ruta	Descripción	Auth
GET	/tasks	Obtener todas las tareas del usuario	Sí (JWT)
POST	/tasks	Crear nueva tarea	Sí (JWT)
PUT	/tasks/:id	Editar texto, estado o fecha	Sí (JWT)
DELETE	/tasks/:id	Eliminar tarea	Sí (JWT)

Todas las rutas de tareas requieren el header: Authorization: Bearer <token>

6. Flujo de Autenticación

Login local (usuario + contraseña)

- El usuario envía credenciales a POST /auth/login
- El backend verifica la contraseña con bcrypt
- Si es correcta, genera un JWT con expiración de 7 días
- El frontend almacena el token y lo envía en cada petición

Login con Google (OAuth 2.0)

- El usuario accede a GET /auth/google
- Passport redirige al flujo de autenticación de Google
- Google retorna al callback con el perfil del usuario
- Si el usuario no existe, se crea automáticamente en la base de datos
- Se genera un JWT y se redirige al frontend con el token en la URL

El middleware `auth.middleware.js` valida el JWT en cada petición protegida, extrae el `userId` y lo adjunta al objeto `request` para que los controladores lo utilicen.

7. Estructura del Proyecto

Backend

```
Backend/
├── server.js           → Entrada: middlewares, rutas, Passport
├── database.js        → Conexión SQLite y creación de tablas
├── Controllers/
│   ├── auth.controller.js → Lógica de register, login, googleCallback
│   └── task.controller.js → CRUD completo de tareas
└── Middleware/
    └── auth.middleware.js → Verificación del JWT
```

Frontend

```
src/
├── App.jsx / Home.jsx
├── services/
│   └── taskService.js → Llamadas HTTP al backend
├── store/
│   ├── useAuthStore.jsx → Estado global de autenticación
│   └── useTaskStore.jsx → Estado global de tareas
└── components/
    ├── Login.jsx
    ├── atoms/           → Button, Input, Checkbox, CalendarPicker
    ├── molecules/       → FilterBar, TaskInput, TaskItem
    └── organisms/       → TaskList, CalendarPanel
```

El frontend sigue el patrón de diseño atómico: los componentes se organizan de menor a mayor complejidad (átomos → moléculas → organismos).

8. Decisiones de Diseño

- SQLite fue elegida por su simplicidad: no requiere un servidor de base de datos separado.
- JWT con expiración de 7 días para mantener la sesión sin necesidad de refresh tokens.
- bcrypt para hashear contraseñas antes de almacenarlas; nunca se guarda texto plano.
- Los usuarios de Google no tienen contraseña en la BD (`password = NULL`).
- Las tareas tienen aislamiento por usuario: cada consulta filtra por `user_id` y verifica propiedad antes de editar o borrar.

